SOURCE REVIEW / CANDIDATE HARDENING

Dooit security audit & remediation

Technical assessment of the supplied Dooit 3.3.4 source archive

Prepared 10 October 2026 • Local terminal application • Revision 1

Outcome: patches prepared; runtime assurance pending

The review identified local data-loss and trust-boundary risks, filesystem weaknesses, a cross-workspace ordering bug, and CI/release control gaps. Eight grouped findings have candidate source changes; one release supply-chain finding remains open. No runtime tests were completed.

Executive summary

• No direct application TCP/UDP listener calls were found in the core source. A hardcoded Python network-deny policy and disabled documentation-server scripts were added as defense in depth.

• Migration now has pre-change backups and a row-level transaction. External Python configuration is disabled during normal hardened startup.

• Filesystem path checks, POSIX database modes, import-path restoration and CI triggers were tightened. Regression tests were supplied but not executed.

• The named DevSecOps checklist PDF was not supplied. This report is not a checklist compliance certificate or a production approval.

Assessment boundary

Only the attached source archive and report-design instructions were available. No deployed host, user database, cloud account, GitHub settings, or live port evidence was provided. Project execution was blocked; approval to run isolated tests was requested and remained pending.

Original archive SHA-256

```
3d4d177ec4f82ca500937e52c8f215e4
dbebd3f6732338715c4146783ab2c8ec
```

01 / Scope, evidence & network controls

Method and confidence

The RAR was extracted with path and entry-type checks into a separate review folder. Current application entry points, storage, migration, plugins, hooks, tests, docs scripts and CI workflows were inspected. Findings are supported by source paths and original line references. Severity is contextual engineering judgment, not a CVSS score.

Static checks only

All modified Python files were parsed with Python ast.parse; TOML and package JSON were structurally parsed. The original archive was preserved. A targeted current-file secret-pattern search found no matches; Git history and broad secret detection were not assessed.

Surface	Observed evidence	Change / assurance limit
Core todo application	No direct IP socket/server calls found in dooit/.	Python audit guard added; no live listener test.
Docs development/preview	VitePress scripts can start a local server when invoked.	Scripts now fail closed; direct external invocations not controlled.
Executable user config	Automatic top-level Python execution.	Normal startup restricted to bundled defaults.
Host ports / firewall	No host or deployment access supplied.	No ports closed on a user host; no firewall changed.

Hardcoded deny policy

Normal CLI entry and direct TUI construction install a process-wide audit hook. It denies creation of IPv4/IPv6 sockets, IP bind/connect calls and audited DNS lookups. TCP and UDP share the same deny decision. There is no allowlist, environment override or CLI enable switch. Local Unix-domain sockets are allowed.

This is not OS-level isolation

Native extensions, subprocesses and programs outside the guarded interpreter are not contained. Library-only use outside normal startup is not guaranteed to install the policy. Opening URLs in the app is disabled, but unrelated browsers/services are not controlled. Verify live process sockets and separately enforce OS-level network restrictions where required.

Checklist limitation

“DevSecOps Checklist (Expanded Edition).pdf” was not attached and was not present in the archive. No requirement IDs, score or control-level pass/fail result can be supplied until that document is provided. Generic DevSecOps categories later in this report are a provisional index only.

02 / Data preservation & filesystem paths

F01 | Migration can erase or partially replace existing todos

Severity: High / Source patch supplied; runtime migration and restore tests pending.

Evidence: dooit/backport/migrate_from_v2.py:11, 75-93, 97-141; dooit/utils/database.py:5-10; dooit/api/manager.py:53-58

Problem & impact. Migration opens a database at import time, commits deletion before reading and converting all legacy data, and saves each item in separate commits. A malformed later record can leave the old rows erased and only some new rows saved. The old YAML backup uses a fixed .bak name that can collide with an earlier backup. User data loss during an explicitly confirmed migration. This is a local data-integrity risk, not remote exploitation.

```
if confirm.lower() == "y":
    delete_all_data(manager.session)
# data = self.load_old() occurs later
```

Source remediation. Removed the import-time connection. Migration now reads the top-level YAML first, creates unique owner-only legacy and SQLite backups before row replacement, and wraps delete/save operations in one Manager.transaction(). Failed conversions roll back row changes. Legacy cleanup happens only after commit and reports cleanup failures separately.

Remaining limit. Schema/file creation is not rolled back with row changes. A failed first migration can leave an empty database file. Full nested schema validation, YAML size/depth limits, concurrent-writer handling and backup-restore exercises are still needed.

F02 | Stylesheet identifiers can escape the cache directory

Severity: Medium / Source patch supplied; traversal/symlink regression tests added, not run.

Evidence: dooit/utils/css_manager.py:73-94

Problem & impact. Caller-provided stylesheet IDs are joined directly into file paths. An ID such as ../escape can target a .tcss file outside the intended stylesheets directory. The same problem affects deletion. Existing symlinks can also redirect writes. A local caller or trusted Python extension can overwrite/delete reachable .tcss files. No remote or unauthenticated path to this API was found.

```
css_file = self.stylesheets / f"{uuid}.tcss"
with open(css_file, "w") as f:
    f.write(css)
```

Source remediation. All ID-based operations now use one validator allowing only 1-128 letters, digits, underscores or hyphens. Final-component symlinks are rejected. New injections use exclusive creation and O_NOFOLLOW where available. Reads accept only regular .tcss files.

Remaining limit. Duplicate stylesheet IDs now fail rather than overwrite. The shared cache design and main dooit.tcss write path were not fully redesigned; same-user filesystem races and concurrent app instances remain outside the fix.

03 / Configuration trust & private storage

F03 | User configuration is executable Python

Severity: High / conditional / Hardening source patch supplied; startup behavior test pending.

Evidence: dooit/ui/api/plug.py:40-45; dooit/ui/api/loader.py:28-40; dooit/__main__.py:11-20

Problem & impact. Startup automatically imports the user config.py and executes its top-level code. Explicit --config does the same. This is an intended extension feature, but unsafe if a user runs a downloaded config or another actor can modify the config file. An untrusted configuration can act with the user's permissions, including reading files or launching child processes. Severity depends on the trust of the config; the application is not a network service.

```
load_file(self, self.config)
...
spec.loader.exec_module(module)
```

Source remediation. The hardened startup scans only the bundled defaults. CLI and direct TUI construction reject external config paths. Automatic external URL opening is also disabled for the offline edition.

Remaining limit. This intentionally removes custom config/plugin convenience from normal startup. The internal loader API is still executable code and must remain trusted. This is not a sandbox for malicious Python or native extensions.

F04 | Database privacy depends on the process umask

Severity: Medium / Source patch supplied; POSIX permission/symlink tests added, not run.

Evidence: dooit/api/_vars.py:4-8; dooit/api/manager.py:23-29

Problem & impact. Default directories and SQLite files are created without explicit owner-only modes. With a typical 022 umask, the data directory/file may be readable by other local users when parent-directory access permits. Symlink database paths are not rejected. Potential disclosure of plaintext todo contents to other users on the same computer. Actual exposure depends on deployment permissions; no user database was supplied.

```
DATABASE_FILE.parent.mkdir(parents=True, exist_ok=True)
self.engine = create_engine(f"sqlite:///{path}")
```

Source remediation. Added private storage helpers. The application-owned data directory is 0700 on POSIX; database files are precreated/tightened to 0600. Ownership, parent write permissions, regular-file type and final symlinks are checked. SQLite URLs are constructed with URL.create instead of string interpolation.

Remaining limit. Windows ACLs and encrypted storage are not implemented. These checks do not replace an OS sandbox. Parent/ancestor trust, SQLite sidecar behavior and migration backup permissions need deployment verification. Existing default-directory permissions will be tightened.

04 / Import integrity & correctness

F05 | Configuration loader changes the global import search path

Severity: Medium / Source patch supplied; normal/exception restoration test added, not run.

Evidence: dooit/ui/api/loader.py:18-25, 32-36

Problem & impact. register() appends a config directory before entering the temporary context. The context removes a matching path even when that entry existed before the call. Exceptions and duplicate entries can leave or remove unexpected entries in global sys.path. Import resolution can change after loading config; nearby modules may shadow expected imports. This is an import-integrity and reliability issue, conditional on trusted config use.

Source remediation. Removed the extra append. The context snapshots sys.path and restores the same list contents in finally, including when a loaded module raises an exception.

Remaining limit. sys.path is process-global. Concurrent module loading is not synchronized; external Python execution remains trusted-only and is disabled in normal hardened startup.

F06 | Inserting an item shifts unrelated sibling groups

Severity: Medium / Source patch supplied; cross-parent ordering tests added, not run.

Evidence: dooit/api/hooks/fix_hooks.py:15-24, 33-42

Problem & impact. Both insertion hooks update every row whose order_index is high enough, without restricting the update to the inserted item's parent. Adding an item under workspace A can change ordering values under workspace B. Cross-workspace ordering corruption. This is a business-logic/data-integrity bug, not SQL injection; the original values are parameterized.

Source remediation. Workspace updates now filter by parent_workspace_id. Todo updates filter by both parent_workspace_id and parent_todo_id using null-safe SQLite IS comparisons. Updates remain parameterized.

Remaining limit. Concurrent insertion and nested-copy ordering have not been stress-tested. SQLAlchemy relationship/FK timing must be confirmed by the regression suite.

F07 | Input and fallback handling has reliability edge cases

Severity: Low / Source patch supplied; date regression test added, not run.

Evidence: dooit/utils/date_parser.py:6-10; dooit/utils/css_manager.py:65-69; dooit/api/manager.py:41-46

Problem & impact. Date parsing catches only ParserError, so overflow errors can escape. Unknown theme names store a class instead of an instance. Manager.has_changed() expires the global manager session rather than the current instance. Malformed large dates can interrupt UI processing. Inconsistent fallback types and the wrong session target can break integrations or independent Manager instances.

Source remediation. Date parsing catches ValueError and OverflowError and returns an invalid-input result. The theme fallback uses a DooitThemeBase instance. Database change detection now expires self.session.

Remaining limit. These are targeted edge-case fixes, not an exhaustive UI or parser correctness review. Additional timezone, recurrence and multi-instance tests are recommended.

05 / CI quality gates & release supply chain

F08 | Test/lint CI does not gate pull requests or main

Severity: Medium / Workflow source updated; no GitHub jobs executed and no branch rules changed.

Evidence: .github/workflows/pytest.yml:3-10, 22-26; .github/workflows/ruff.yml:4-11, 26-29; .github/workflows/vitepress_deploy.yml:38-41

Problem & impact. Test and lint workflows run only for pushes to develop; pull_request triggers are commented out. Dependency setup is not explicitly lock-enforcing, and Ruff is installed without a version. Docs build uses bun install without frozen-lock enforcement. Changes can reach main without repository-defined test/lint checks. Builds may drift from the checked-in dependency resolutions.

```
push:
  branches:
    - develop
# pull_request:
```

Source remediation. Enabled main/develop push and pull-request triggers with read-only token permissions for test/lint workflows. Tests use uv sync/run --locked; Ruff is pinned to the existing lock version 0.13.1; docs installs use --frozen-lockfile.

Remaining limit. Require these checks through repository branch protection/rulesets. GitHub permissions and actual job behavior must be verified. The binary release pipeline remains separate and unremediated.

F09 | Release dependencies and workflow actions are mutable

Severity: Medium / OPEN - owner-approved supply-chain changes required.

Evidence: .github/workflows/app.yml:18-35, 59-68; .github/workflows/python-publish.yml:21-37; .github/workflows/vitepress_deploy.yml:21-34; pyproject.toml:17-41

Problem & impact. Release packaging installs a Git repository without a commit pin and resolves packages from ranges. Most action references use mutable version tags rather than full commit SHAs. Publishing uses a long-lived secret token. Test/lock checks are not release prerequisites. A changed upstream dependency or action can alter a release without changes to this repository. Exposure is a supply-chain possibility, not proof of compromise or a confirmed dependency CVE.

```
pip install git+https://github.com/dooit-org/dooit-extras.git
uses: softprops/action-gh-release@v1
UV_PUBLISH_TOKEN: ${{ secrets.UV_PUBLISH_TOKEN }}
```

Source remediation. No unverified action SHA or upstream commit was invented. This finding is intentionally left open. Verify and pin action SHAs and the extras commit, use locked release dependency inputs, enforce protected release gates, and evaluate PyPI trusted publishing with a protected environment.

Remaining limit. No live vulnerability database scan, PyPI audit, npm/Bun audit, full Git-history secret scan, SBOM generation, signing or provenance validation was completed. Existing lock hashes do not establish absence of vulnerabilities.

06 / Validation ledger & unresolved controls

What was actually completed

Check	Result / confidence
Archive identity and source review	Completed. SHA-256 recorded; original source retained.
Candidate source syntax / structures	Python AST, TOML and JSON parsing completed; no application imports.
Baseline / regression / UI / migration tests	NOT RUN. Project execution blocked; approval pending.
Actual sockets, listeners, firewall state	NOT ASSESSED. No live deployment or port scan.
GitHub workflows / branch protection	NOT RUN / NOT CHANGED. Source edits only.
Dependency CVEs, docs audit, complete secrets scan	NOT COMPLETED. No claim of vulnerability-free dependencies.
Exact requested checklist mapping	BLOCKED BY MISSING INPUT. Checklist PDF required.

New regression coverage supplied, not executed

Tests target unsafe stylesheet IDs and symlinks; normal/exception sys.path restoration; IPv4/IPv6 TCP/UDP denial; DNS denial; bundled-only config scan; POSIX file modes and database symlinks; date overflow; cross-parent workspace/todo ordering; and atomic row rollback. These are proposed tests, not passing evidence. Existing tests remain in the archive.

Provisional DevSecOps control index

Category	Current status
Input / code / integrity	Targeted patches supplied; full runtime assurance pending.
Data protection / recovery	POSIX hardening and migration changes; restore/Windows checks pending.
Network exposure	Static core search plus candidate Python guard; OS/live checks pending.
CI / change control	Test/lint triggers edited; required checks need owner action.
Dependencies / release provenance	Release-chain issues open; SBOM, CVE scan and provenance pending.
IAM / deployment / monitoring / incident response	Not assessed: no operational evidence supplied.

07 / Safe rollout & delivery manifest

Before deployment

• Preserve the original archive; back up real DB/YAML/config files. Review the candidate in a separate folder.

• Use disposable data and a temporary HOME/XDG environment. Obtain execution approval and install locked dependencies.

• Run tests, lint/format, TUI operations and migration/restore cases. Verify each supported OS separately.

• Check live listeners and child processes. Use OS isolation for stronger guarantees; avoid blanket firewall changes.

• Require GitHub checks; verify and pin release inputs; protect publishing credentials and environments.

• Supply the checklist and resolve outstanding controls before security sign-off.

Suggested checks — not executed here

```
uv sync --locked
uv run --locked pytest
uv run --locked pytest tests/security/test_hardening.py -q
uv run --locked ruff check .
uv run --locked ruff format --check .
```

Behavior changes to accept

Custom Python startup config, Python IP networking, app URL opening and docs-server scripts are disabled. Duplicate stylesheet IDs fail and default POSIX storage modes tighten. Existing extensions may break; review the Markdown compatibility notes.

Deliverables

File	Purpose
dooit-security-audit.pdf	Technical findings, evidence, changes and validation limits.
dooit-findings-and-fixes.md	Plain-language explanation and verification steps.
dooit-hardened-source.zip	Candidate source and unrun tests; Git history excluded.
dooit-hardening.patch	Unified source diff against the supplied working tree.
audit-manifest.json	Archive/source hashes, changed-file inventory and scope.

Conclusion

Eight grouped findings have source changes; release-chain controls remain open. Runtime validation, host-port closure and exact checklist compliance are not verified.